

# P0779R0: Proposing `operator try()` (with added native C++ macro functions!)

Document #: P0779R0  
Date: 2017-10-15  
Project: Programming Language C++  
Evolution Working Group  
Reply-to: Niall Douglas  
<[s\\_sourceforge@nedprod.com](mailto:s_sourceforge@nedprod.com)>

Something which would be useful to the Expected proposal [P0323], the C++ Monadic Interface proposal [P0650] and the proposed Boost.Outcome library <https://ned14.github.io/outcome/> would be if we could customise the `try` operator in a similar way to how Swift<sup>1</sup> and Rust<sup>2</sup> implement `try`. This saves having to type so much tedious boilerplate when writing code with Expected all the time.

Example in code:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 // Without operator try 2 template&lt;class T&gt; using expected = 3   std::expected&lt;T, std::error_code&gt;; 4 5 expected&lt;int&gt; get_int() noexcept; 6 7 expected&lt;float&gt; get_float() noexcept 8 { 9   expected&lt;int&gt; _int = get_int(); 10 11   // If get_int() failed, propagate the error 12   if(!_int) 13     return unexpected(_int.error()); 14   float ret = (float) *_int; 15 16   // If the float cannot wholly represent 17   // the int, return an error 18   if((int) ret != *_int) 19     return unexpected(std::errc:: 20       result_out_of_range); 21 22   // Otherwise return success 23   return ret; 24 }</pre> | <pre>1 // With operator try 2 template&lt;class T&gt; using expected = 3   std::expected&lt;T, std::error_code&gt;; 4 5 expected&lt;int&gt; get_int() noexcept; 6 7 expected&lt;float&gt; get_float() noexcept 8 { 9   int _int = try get_int(); 10 11 12 13   float ret = (float) _int; 14 15   // If the float cannot wholly represent 16   // the int, return an error 17   if((int) ret != _int) 18     return unexpected(std::errc:: 19       result_out_of_range); 20 21   // Otherwise return success 22   return ret; 23 }</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

<sup>1</sup>The `try` keyword in Swift ([https://developer.apple.com/library/content/documentation/Swift/Conceptual/Swift\\_Programming\\_Language/ErrorHandling.html](https://developer.apple.com/library/content/documentation/Swift/Conceptual/Swift_Programming_Language/ErrorHandling.html)).

<sup>2</sup>The `try!` macro in Rust (<https://doc.rust-lang.org/std/macro.try.html>).

In other words, we just want to ‘inject’ some type-specific boilerplate into the calling scope in a similar way to how the [N4680] Coroutines TS implements `co_await`.

# 1 Motivation

## 1.1 Frequency of use

Those who have not programmed in Rust nor Swift, and are not practised in writing code which uses `Expected|Outcome` extensively, are not aware how frequently one performs the `try` operation.

With C++ exception handling, the points at which control flow can change are *invisible*. This is not the case with `Expected|Outcome` code where the programmer must explicitly annotate each potential control flow change point with either explicit `if` logic, or a `try`. For obvious reasons, these rapidly proliferate and become tedious to constantly write, so programmers will seek shortcuts to avoid constantly writing the same boilerplate again and again.

Due to such frequency of use, without language support for `try`, one would probably reach for a C macro expanding into a GCC/clang language extension called ‘statement expressions’<sup>3</sup>. Here is one potential implementation<sup>4</sup>:

```
1 #define TRYX(m) \
2   ({ \
3     auto res = (m); \
4     if(!res.has_value()) \
5       return unexpected(res.error()); \
6     res.value(); \
7   })
```

The use of C macros is not ideal. The use of a non-standard language extension is worse again. This has bothered some people enough to seek workarounds by misusing the C++ language.

---

<sup>3</sup><https://gcc.gnu.org/onlinedocs/gcc/Statement-Exprs.html>

<sup>4</sup>Yes, it is incorrect because it doesn’t handle non-moveable and non-copyable type `T`. It has been simplified for brevity of presentation here.

## 1.2 Failure to standardise this means people may abuse `co_await` to achieve the same thing

In September 2017, Facebook Folly’s `Optional` gained the ability to be awaited upon with `co_await`<sup>5</sup>. This is not a true coroutine `await`, rather it’s an abuse of awaiting to inject boilerplate due to the C++ language’s inability to otherwise do this at the language level. Quoting this code example from a CppCon 2017 talk called ‘Coroutines: What can’t they do?’ by Toby Allsopp<sup>6</sup>:

|                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 optional&lt;vector&lt;double&gt;&gt; 2 parse_vector(istream&amp; s) { 3     optional&lt;int&gt; n = parse_int(s); 4     if(!n) return (); 5     vector&lt;double&gt; result; 6     for(int i = 0; i &lt; *n; ++i) { 7         optional&lt;double&gt; x = parse_double(s); 8         if(!x) return {}; 9         result.push_back(*x); 10    } 11    return result; 12 }</pre> | <pre>1 optional&lt;vector&lt;double&gt;&gt; 2 parse_vector(istream&amp; s) { 3     int n = co_await parse_int(s); 4 5     vector&lt;double&gt; result; 6     for(int i = 0; i &lt; *n; ++i) { 7 8         result.push_back(co_await parse_double(s)); 9     } 10    co_return result; 11 } 12 }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

There are some obvious big problems with this type of abuse of `co_await`. As soon as `co_await` appears in a function, it becomes *coroutinised* and is no longer a normal function. So, you must return via `co_return`, the function’s return type must be an awaitable, and the function returning infrastructure must become owned and managed by coroutines. Furthermore, you cannot await on something different to the coroutine return type e.g. a `future<std::string>` when the coroutine returns a `future< expected<int> >`.

For these reasons and more, I find this misuse of `co_await` troubling, and I hope so do you as well. This needs to be nipped in the bud before it goes septic and starts appearing across the C++ ecosystem due to lack of a better alternative.

## 2 Solutions

I will propose two potential solutions to the problem of injecting the necessary type-specific boilerplate for an `operator try`: (i) a narrow proposal and (ii) a wide proposal.

### 2.1 Implement `operator try` just like `operator co_await`:

```
1 template <class T, class E>
2 constexpr auto operator try(std::expected<T, E> v) noexcept
3 {
4     struct tryer
```

<sup>5</sup><https://github.com/facebook/folly/blob/master/folly/Optional.h>

<sup>6</sup>[https://www.youtube.com/watch?v=mlP1MKP8d\\_Q](https://www.youtube.com/watch?v=mlP1MKP8d_Q), about 30 mins in.

```

5  {
6      std::expected<T, E> v;
7
8      constexpr bool try_return_immediately() const noexcept { return !v.has_value(); }
9      constexpr auto try_return_value() { return std::move(v).error(); }
10     constexpr auto try_value() { return std::move(v).value(); }
11 };
12     return tryer{ std::move(v) };
13 }
14
15
16 // Introductory example expanded
17 template<class T> using expected = std::expected<T, std::error_code>;
18
19 expected<int> get_int() noexcept;
20
21 expected<float> get_float() noexcept
22 {
23     int _int = try get_int(); /* --> auto __unique = operator try(get_int());
24                               if(__unique.try_return_immediately())
25                                   return __unique.try_return_value();
26                               _int = __unique.try_value();
27     */
28
29     float ret = (float) _int;
30
31     // If the float cannot wholly represent
32     // the int, return an error
33     if((int) ret != _int)
34         return unexpected(std::errc::result_out_of_range);
35
36     // Otherwise return success
37     return ret;
38 }

```

If implementing `co_await` using the same design pattern as the above is uncontroversial, then I guess so is the above. It solves the direct problem at hand quickly and simply.

But can we solve this whole class of injecting boilerplate problems in one fell swoop, now and forever?

## 2.2 Implement `operator try` by adding native C++ macro functions to the language

This section likely could form a paper of its own ☺. If you like the idea, please do feel free to submit a P-paper proposing it. I'm no language person, I'm the wrong one to propose it seriously.

Operator `try` is hitting the exact same problem as the Coroutines TS ran into when implementing `co_await`: **boilerplate injection**. C++'s current method of injecting boilerplate is the C preprocessor, and it is non-ideal for a long list of reasons which is why the Coroutines TS adopted its solution which looks exactly like our solution in the preceding section.

But what if C++ had a dedicated language feature for injecting boilerplate? Rust has a feature like this which it calls ‘macros’<sup>7</sup>. These are normal functions, but their contents are injected into the scope of the point of invocation. Indeed, Rust’s `try!` operation is implemented using just such a macro (one can distinguish macro functions in Rust by the bang token ‘!’ at the end of the function name).

Could we perhaps implement the same thing in C++? Well we can’t use the bang token ‘!’ because `return!(v);` might mean ‘inject contents of the `return!` macro function’ or it might mean ‘return logical NOT of `v`’, and the same rationale applies to all C++ operator tokens except possibly for ‘?’ and ‘.’.

But it turns out that the ‘#’ token is available to us: the C preprocessor must emit a ‘#’ token if it is not the first non-whitespace token in a line and is not inside a parameterised macro definition. Moreover, GCC, clang and MSVC all error out about stray ‘#’ tokens if they leak into the preprocessor output. Therefore, no valid code is out there using the ‘#’ token in identifier names, and is available to us for this use case.

*[One issue I don’t know if it is important or not is that the C preprocessor when tokenising the input will treat identifiers as not being able to have ‘#’ in them, so `#define foo# boo` will generate a preprocessor error. There are already quite a few instances where C and C++ tokenise an input differently, but how important it is that the C preprocessor can token swap the names of these proposed C++ macro functions I don’t know]*

So what would one of these native C++ macro functions look like?

```
1 // Macro functions look just like a free function except for the # at the
2 // end of the function name. Note that the # counts as part of the identifier,
3 // so return# does not collide with the return keyword.
4 template<class T>
5 inline int return#(T v)
6 {
7     if(v > 0)
8         return -> v; // control flow keyword + '->' means it affects the
9                     // calling function, not this macro function
10    if(v < 0)
11        break ->;    // Also this break is executed in the calling function
12                    // If break isn't valid at the point of use
13                    // in the calling function, it will not compile
14
15    // We can inject variable declarations into the calling function
16    // with typename + '->'. This is useful for RAII triggered cleanup
17    // i.e. these get destructed when the scope of the call point exits.
18    // Note that the actual name of the variable injected will
19    // be some very unique identifier which cannot collide with any
20    // other variable, including those injected by other macro functions
21    int -> a = 5;
22
23    // Otherwise this function macro has a local scope, and code
24    // executed here remains here
25    size_t n = 0;
26    for(; n < 5; n++)
27    {
```

---

<sup>7</sup><https://rustbyexample.com/macros.html>

```

28     // We can also refer to variables previously injected into the
29     // caller's scope by this macro function like this.
30     // This lets one keep state across invocations of the macro function
31     (-> a) ++;
32 }
33
34 // This returns a value from this function macro to the caller
35 // If you wrote return -> a, that would be a compile error
36 // as there is no variable called a in this scope.
37 return (-> a);
38 }

```

To summarise, macro functions are like normal free functions except that they are attached to the scope of their calling non-macro function. They can inject control flow into their calling function by placing the ‘->’ token after a C++ keyword, instantiate variables into a per-macro-function scope with the same lifetime as the calling non-macro-function’s scope, and retrieve/modify variables in that per-macro-function scope previously instanced.

And in action:

```

1  static int values[]; // some array somewhere
2  int foo(int n)
3  {
4      int acc = 0;
5      for(;;)
6      {
7          // Macro functions are invoked just like normal functions,
8          // no token pasting. So n will be incremented exactly once.
9          acc += return#(values[n++]);
10     }
11 }

```

My personal, not fully thought through, recommendations for these native C++ macro functions:

- Macro functions are namespaced, overloadable and participate in ADL etc just like ordinary free functions. This lets you write `foo#(something)` and ADL will find a most appropriate `foo#` macro function overload.
- Macro functions must always be fully defined at the point of invocation in the current translation unit (Modules notwithstanding). They cannot be `extern`.
- Macro functions must always be `inline`, though as they can’t have linkage anyway, this is a moot point.
- You cannot take the address of a macro function. It does not exist as a free standing function.
- Their identifiers do however identify a unique type each, and thus can be passed around wherever a template type would be allowed. You can thus supply them to templates e.g.

```

1  template<template<class> class ReturnMacro#> int foo(int n)
2  {
3      int acc = 0;
4      for(;;)
5      {

```

```

6      // Macro functions are invoked just like normal functions,
7      // no token pasting. So n will be incremented exactly once.
8      int ret = ReturnMacro#<int>(values[n++]);
9
10     // int a was injected here by the macro function
11     acc += a;
12 }
13 }
14 ...
15 foo<return#>(5);

```

Note how the template parameter's name must also end with a '#', otherwise it wouldn't compile.

- Macro functions can of course call other macro functions. The -> injection of control flow or variables always happens to the non-macro-function caller, not to any macro function caller.
- If a set of macro functions need to share the same injected state, simply use an injected state accessor macro function for retrieving a lvalue reference to it as the injected variables are permuted by the macro function's fully qualified identifier, so the same macro function always sees the same injected variable namespace and no other macro function sees another's.

### 2.2.1 Implementing `co_await` using these native C++ macro functions

As a test to see if they are fit for purpose, let's see how we might implement `co_await` using these.

`auto ret = co_await awaitable_expr;` is effectively this pseudo-code:

```

1  auto __unique = awaitable_expr;
2
3  // Is the awaitable in __unique not ready?
4  if(!__unique.await_ready())
5  {
6      /* Coroutinised functions are compiled twice into non-detached
7      and detached editions. We enter the non-detached edition, if it ever
8      suspends then resumption is to the detached edition. So imagine
9      that this fictitious intrinsic suspends execution, resuming
10     in the detached edition of this function */
11     coroutine_handle<promise_type> coro = __builtin_coro_suspend(resume_detached);
12
13     // Tell the awaitable we have suspended
14     __unique.await_suspend(coro);
15
16     // Return a future to me from the non-detached edition
17     return coro.promise().get_future();
18
19 resume_detached:
20     // When it returns here we are resumed into the detached edition
21 }
22
23 // Ask the awaitable for the value to emit from the co_await operator
24 auto ret = __unique.await_resume();

```

So instead of the complex operator `co_await` currently proposed in the Coroutines TS, we get this instead:

```
1 namespace std { namespace coroutines {
2     inline auto await#(auto awaitable_expr)
3     {
4         if(!awaitable_expr.await_ready())
5         {
6             // affects the calling function, this macro function has no linkage
7             coroutine_handle<promise_type> coro = __builtin_coro_suspend(resume_detached);
8             __unique.await_suspend(coro);
9
10            // Caller returns future to suspended coroutine
11            return -> coro.promise().get_future();
12 resume_detached:
13        }
14        // Macro function returns result from await
15        return awaitable_expr.await_resume();
16    }
17 }}
```

And voilà, `std::coroutines::await#()` nicely replaces `co_await` in a much more flexible, **entirely library defined**, fashion. Similar macro functions can be written for `std::coroutines::yield#()` and `std::coroutines::return#()`. No core C++ language changes with new keywords needed.

## 2.2.2 Implementing ranged for using these native C++ macro functions

Ranged for loops are already in the language, so this is a bit of a moot point. But let's see how one might implement them with macro functions anyway:

```
1 namespace std {
2     inline void item#(auto &&iterable)
3     {
4         // Store the current iterator in my macro local scope
5         // attached to my calling function
6         auto -> it = iterable.begin();
7
8         // Inject a suitable for loop into my calling function
9         for -> (; (-> it) != iterable.end(); ++(-> it));
10
11        // As that was a control flow taking a target, we also
12        // injected a scope around the statement following the
13        // macro invocation.
14    }
15    inline auto &item#()
16    {
17        // We get the same macro local scope as any other item#()
18        // overload in the same namespace
19        return *(-> it);
20    }
21    inline void foreach#(auto &&iterable)
22    {
23        // Only item# can see state it itself injected
24        item#(iterable);
25    }
26 }
```

```

25     }
26 }

```

And a sample of usage:

|                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 // Usage 2 std::vector&lt;std::vector&lt;int&gt;&gt;&gt; c; 3 4 foreach#(c) 5 6 7 8 { 9     auto &amp;row = item#(); 10 11     foreach#(row) 12 13     { 14         std::cout &lt;&lt; item#() &lt;&lt; ", "; 15     } 16 17     std::cout &lt;&lt; std::endl; 18 } 19 // </pre> | <pre> 1 // Expanded 2 std::vector&lt;std::vector&lt;int&gt;&gt;&gt; c; 3 auto __unique1 = c.begin(); 4 for(; __unique1 != c.end(); ++__unique1) 5 // Extra scope injected by macro injection 6 // of for loop 7 { 8     { 9         auto &amp;row = *__unique1; 10        auto __unique2 = row.begin(); 11        for(; __unique2 != row.end(); ++__unique2) 12        { 13            { 14                std::cout &lt;&lt; *__unique2 &lt;&lt; ", "; 15            } 16        } 17        std::cout &lt;&lt; std::endl; 18    } 19 } </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This `foreach#()` implementation has some drawbacks however. The most obvious is that if you nest two `foreach` loops, `item#()` will only return the current item for the innermost loop. This is easily worked around by binding a reference to the current item for outer loops as one goes, but it isn't as clean as with ranged `for` loops built into the language.

That said, there is clearly lots of power and flexibility available here, and without all the problems that C macros have. The EWG I think will see lots more boilerplate injecting requests landing before it in the next few years. Adding a facility like native C++ macro functions could eliminate much of that workload in my opinion, and enable C++ to evolve onwards without being blocked as much on WG21's capacity limits.

### 2.2.3 Implementing `try` using these native C++ macro functions

Let's end this section with implementing `try` for `Expected` using this new mechanism:

```

1 template <class T, class E>
2 inline auto try#(std::expected<T, E> v)
3 {
4     // If there is an error, propagate that error immediately
5     // by injecting an immediate return of the error into my caller
6     if(!v.has_value())
7         return -> std::unexpected<E>(std::move(v).error());
8
9     // Otherwise the output of this macro is the value.

```

```

10     return std::move(v).value();
11 }
12
13
14 // Introductory example expanded
15 template<class T> using expected = std::expected<T, std::error_code>;
16
17 expected<int> get_int() noexcept;
18
19 expected<float> get_float() noexcept
20 {
21     int _int = try_(get_int());
22     float ret = (float) _int;
23
24     // If the float cannot wholly represent
25     // the int, return an error
26     if((int) ret != _int)
27         return unexpected(std::errc::result_out_of_range);
28
29     // Otherwise return success
30     return ret;
31 }

```

Similar to the ranged for example implementation, this isn't *quite* as clean looking as the earlier `operator try`, but it sure beats `TRYX(expr)`.

## 3 Conclusions

### 3.1 Please propose a better solution!

Above I have proposed two methods of implementing `operator try`. One is narrowly scoped, focused and highly uncontroversial at least in the form of implementation. The other has a very wide scope which could help C++ users all over the world avoid the evils of the C preprocessor for injecting boilerplate, a very common use case.

If you think you have a better idea for dealing with boilerplate injection, please do absolutely consider submitting a P-paper here which shows that both of these proposals are an inferior design. I'll be more than interested to see what you propose.

### 3.2 Try chaining and Optional

I have not discussed in this proposal paper 'try chaining' at all. This is a facility whereby the `try` operator can be told to act recursively, repeatedly trying the result of a `try` until a non-tryable result is encountered.

The reason that I have not discussed it here is because recursive unwrapping is definitely the most useful with `Optional`, and not in practice that common with `Expected`|`Outcome` as in my personal experience, one does not frequently actually store variables in unwrapped form like you do with

Optional. The reason that you do not is because an ExpectedOutcome returns an error instead of a value, and more often than not, you want to know about that error sooner rather than later, so you might as well unwrap it. Optional, on the other hand, is a maybe-something, so you tend to use it as a ‘slot’ and thus write logic based on slots being filled or not. I find myself not using Outcome as ‘slots’ in over three years of writing code with it. Maybe that’s just me of course, but I suspect that try chaining really is mostly useful with Optional not Expected. And if so, perhaps a visitor-based library function would be a better solution to a chaining-capable **operator try**. I’ll leave that for readers to decide.

Incidentally, this insight that you won’t use Expected like Optional is one of the key reasons why Outcome deviates in design and focus from Expected, and you can find out more about that in *P0762R0 Concerns about Expected* also in this mailing.

### 3.3 The elephant in the room

Adding any **operator try** at all to the language is of course controversial. You may have noticed my studious avoidance of ever mentioning the phrase ‘lightweight error handling’ in anything mentioned in this paper. If I had mentioned that phrase, I would be compelled to discuss performance, and to both demonstrate and **prove** the performance superiority of Expected/Outcome based error handling over table-based exception throws where failure is not extremely rare.

However error handling performance cannot be proven with cheap-to-implement micro-benchmarks. It requires you to Monte-Carlo multiple real world code bases and to generate lots of statistical profiles. As it happens, I am also a fully trained Economist, and such an Econometric analysis would not be hard to implement. But it would take lots of time, at least a thousand hours to do it properly by my reckoning. And I’m not willing to invest that sort of time without earning for it, I do earn by the hour as a contract programmer after all.

But I am hoping that this paper inspires someone with much more resources available to them than I have to invest in conclusively proving the merit of this approach. After all, the people over at Swift – many of them former WG21 and Boost members I might add – know their stuff. WG21 choosing to replicate their choice for error handling via Expected and **operator try** is likely not a bad choice.

We just need someone with deep pockets, or an awful lot of free time, to generate the statistical proof of the merits this approach so WG21 can sign off on it. In case you are such a person and are wondering just how much work this actually is, John Lakos has been doing talks during 2017 with performance ‘heat maps’ of local allocators i.e. colour coded statistical distributions of performance for various canned use cases with various allocators. It took Bloomberg a considerable investment of manpower and time to generate these. A similar level of investment would be needed here too if one is to generate a conclusive proof sufficient to eliminate all doubt in WG21 as to the merit of this approach.

## 4 Acknowledgements

- Vicente J. Botet Escribá for his extensive commentary on earlier drafts of this paper.
- std-proposals for helping me work through the ‘native C++ macro functions’ idea, particularly Ville Voutilainen and Nicol Bolas.
- Michael Park for making available this LaTeX template at <https://github.com/mpark/wg21/>.

## 5 References

- [P0650] Vicente J. Botet Escribá,  
*C++ Monadic interface*  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0650r0.pdf>
- [P0323] Vicente J. Botet Escribá,  
*A proposal to add a utility class to represent expected object (Revision 4)*  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0323r2.pdf>
- [P0262] Lawrence Cowl, Chris Mysisen,  
*A Class for Status and Optional Value*  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0262r0.html>
- [N4680] Gor Nishanov,  
*C++ Extensions for Coroutines TS*  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4680.pdf>